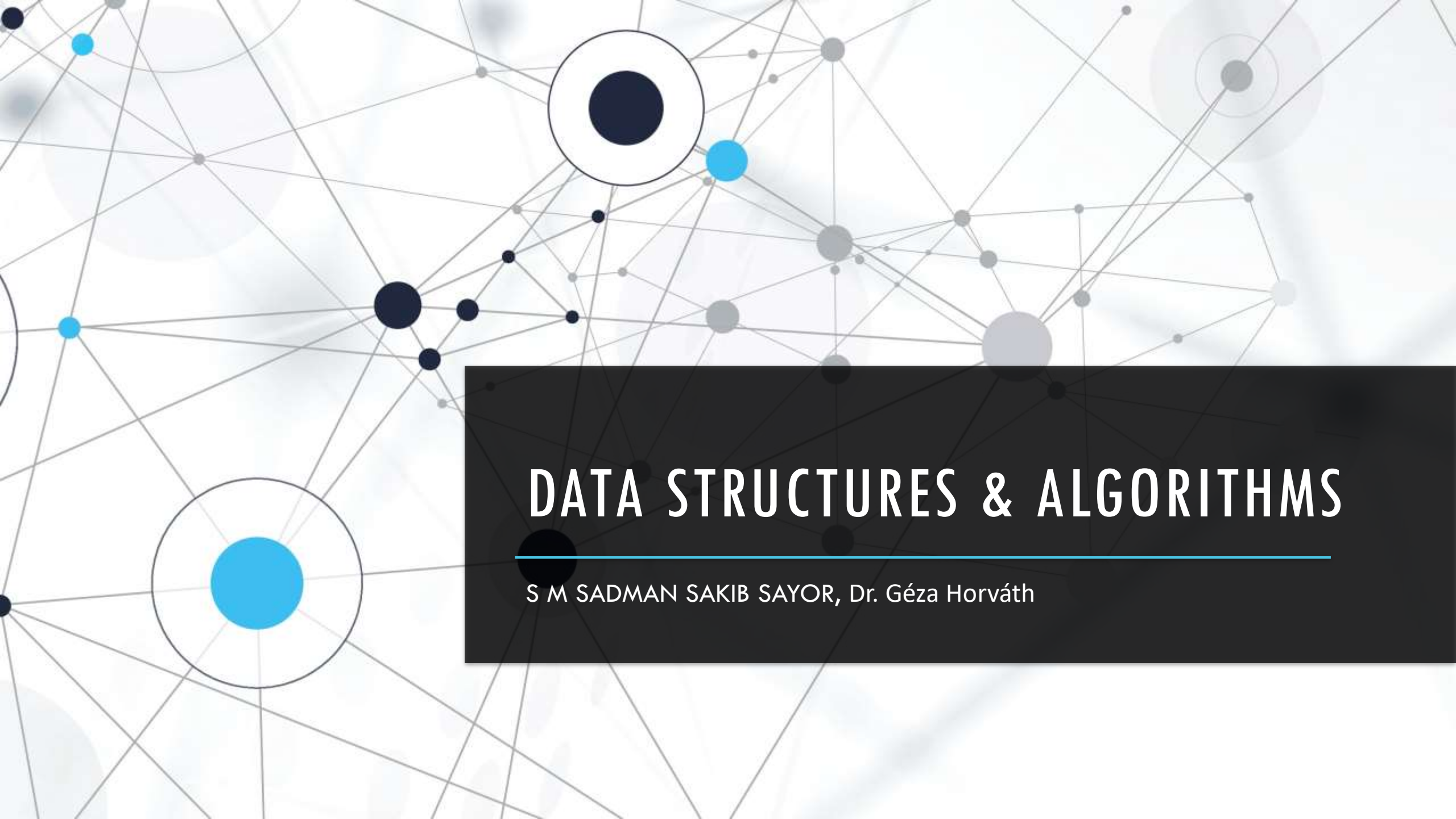


DATA STRUCTURES & ALGORITHMS

S M SADMAN SAKIB SAYOR, Dr. Géza Horváth

SEMESTER CONTENT

First Half

- Introduction to DSA
- Design and Analysis of Algorithm
- Asymptotic Notation
- Set, Multi-set, Array and Matrix
- Stack, Queue and List
- Table and Hash-table

Second Half

- Tree , Binary-tree and BST
- AVL Tree
- Red-black Trees
- Graphs
- Parallel Algorithm
- Discussion for Exam

ANALYSIS OF ALGORITHM

- Time Complexity
- Space Complexity

TIME COMPLEXITY – 1

Misconception

Time Complexity != Time Taken

Time Complexity != Runtime

TIME COMPLEXITY – 2

Time Complexity is a Function like normal mathematical function.

Example: $f(x) = 1$, $f(x) = n + 1$ etc

NOTATION OF TIME COMPLEXITY

Notation is a symbol to represent some kind of value like

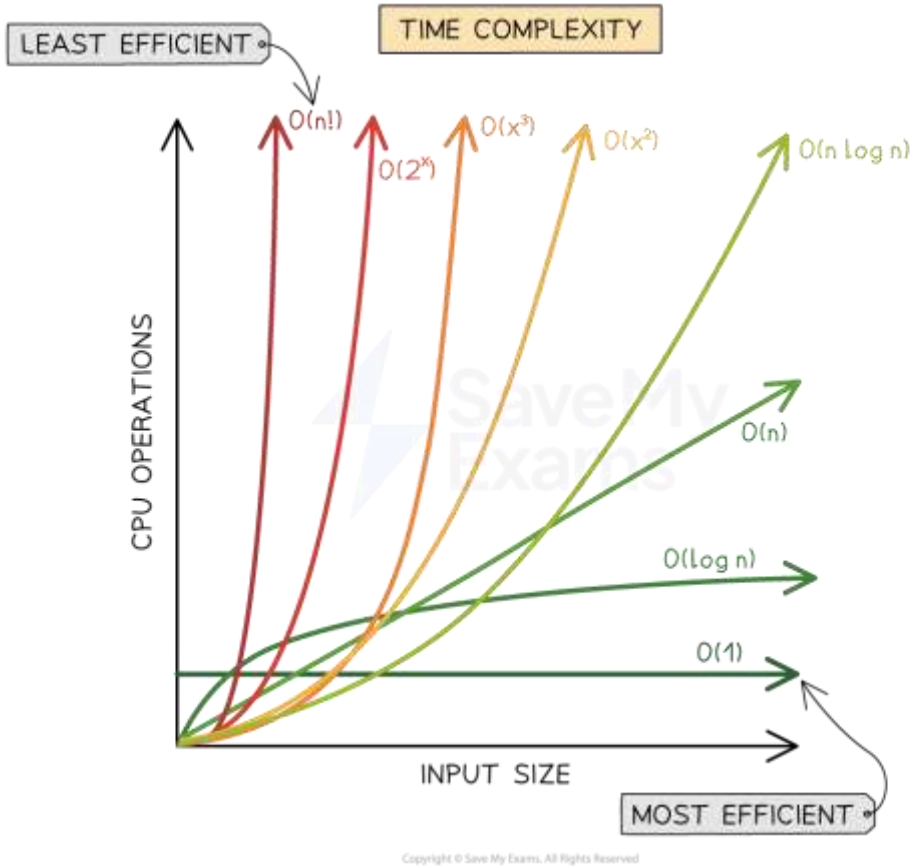
€ 50 here notation is € which represent currency.

<u>Time Complexity Notation</u>

1. O notation – Worst Case
2. Ω notation – Best Case
3. θ Notation – Average Case

BIG O NOTATION

Big O notation describes the upper bound of an algorithm's time or space complexity, representing its worst-case growth rate as a function of input size.



WHY WORST-CASE IS IMPORTANT THAN BEST OR AVERAGE CASE?

- € 10 Million >> € 10

- Website with one million users is more important than website with 100 users.

- Recent Neptun Server down while P.E. Registration because the develpors didn't care about worst case scenrio.

GENERAL RULE

- Igonore Constants

  5n → O(n)

- Certain Terms "Dominate" others

  O(1) < O(log n) < O (n) < O (n log n) < O(n^2)

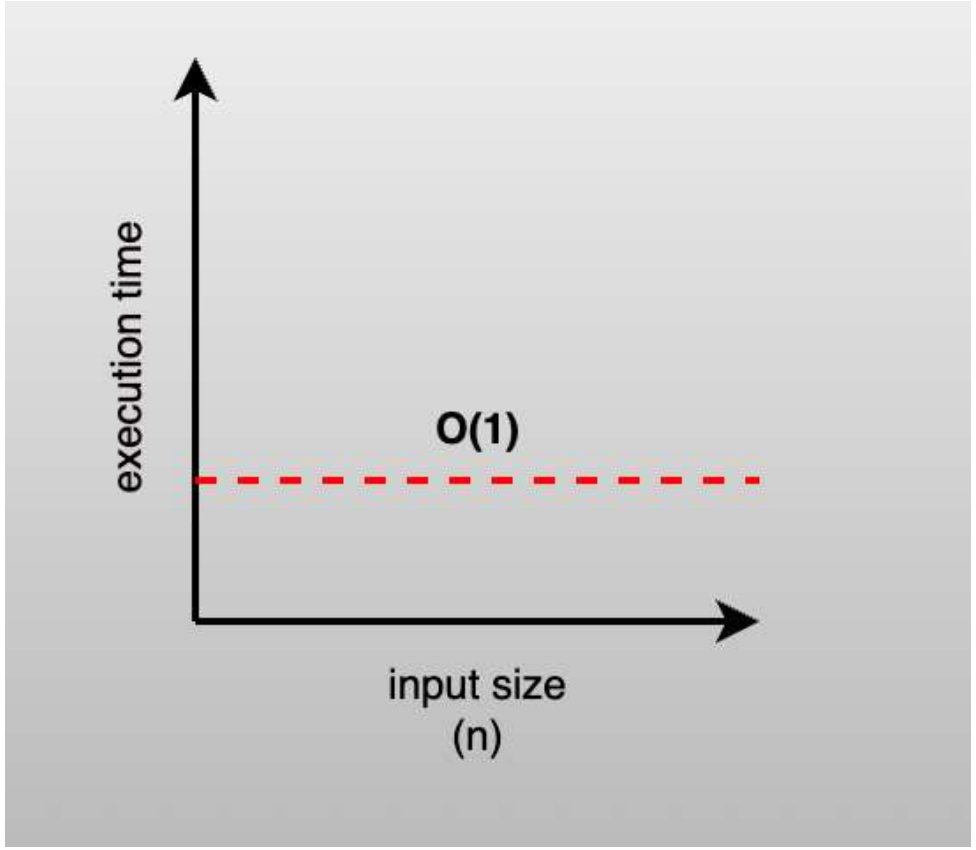
  i.e., ignore low-order terms

EXERCISE - 1

1. f(n) = 12
2. f(n) = 2n + 3
3. f(n) = $3n^2$ + 2n + 1
4. f(n) = 5 log n + 20
5. f(n) = 2n + 3n log n + 19
6. f(n) = $6n^3 + 2n^2$ + 3n + 2

CONSTANT TIME - O(1)

Constant time, O(1), means that the algorithm's execution time or space requirement does not change regardless of the input size.



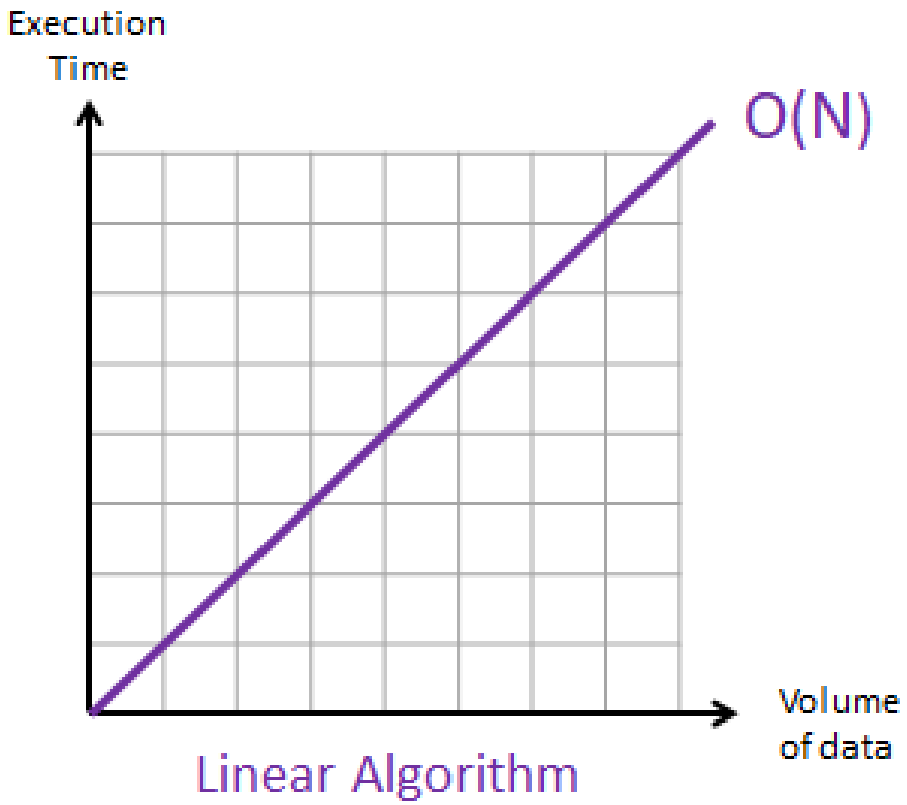
CONCEPT QUESTION -1

```
int i = 5;
int j = 2;
int k = i + j;
```

Find the time complexity?

LINEAR TIME - O(N)

Linear time, O(n), means the algorithm's execution time or space requirement grows directly in proportion to the input size.



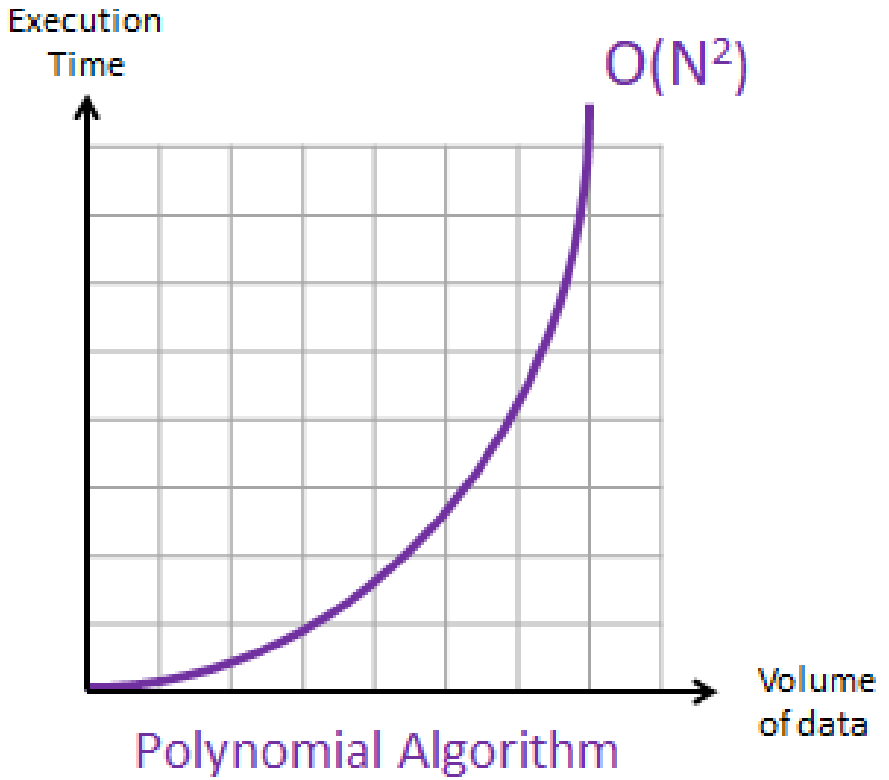
CONCEPT QUESTION - 2

```
for(int i = 1; i <= n; i++)
    {
        x++;
    }
```

Find the time complexity?

QUADRATIC TIME - O(N^2)

Quadratic time, O(n^2), means the algorithm's execution time or space requirement grows proportionally to the square of the input size.



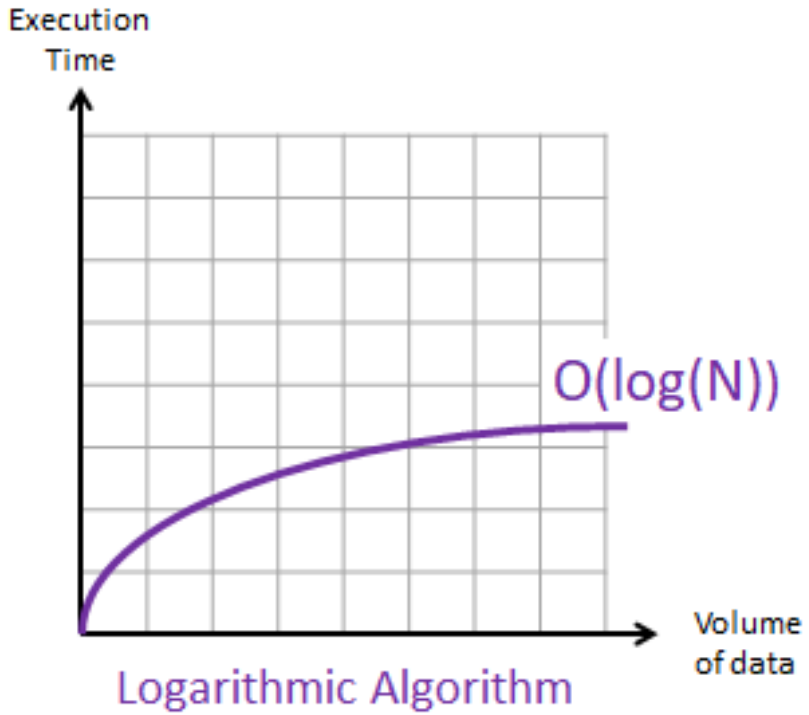
CONCEPT QUESTION - 3

```
for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            x++;
    }
}
```

Find the time complexity?

LOGRARITHMIC TIME - O(LOG N)

Logarithmic time, O(log n), means the algorithm's execution time or space requirement grows in proportion to the logarithm of the input size.



CONCEPT QUESTION - 4

```
int i = n ;
 while (i != 1) {
        i = i / 2;
            }
```

Find the time complexity?

EXERCISE - 2

```
for(int i = 1; i <= n; i++){
        x++;
}
for(int i = 1; i <= n; i++){
        for(int j = 1; j <= n; j++){
            x++;
    }
}
```

Find the time complexity?

EXERCISE - 3

```
int m = 0;
for(int i = 1; i <= n; i++){
        for(int j = 1; j <= 2*i; j++){
            m++;
    }
}
```

Find the time complexity?

EXERCISE - 4

```
int i = n * n ;
 while (i != 1) {
        i = i / 2;
            }
```

Find the time complexity?

EXERCISE - 5

```
int x = 0;
int n;
while( n > (x + 1) * (x + 1)){
    x++;
}
```

Find the time complexity?

EXERCISE - 6

```
for (int i = 0; i < n; i++){
    for (int j = 0; j < m; j++){
        a[i][j] = 0;
    }
}
```

Find the time complexity?

APPLICATION OF TIME COMPLEXITY (LEETCODE)

Here we can the check the real life application of time complexity in Leetcode

https://leetcode.com/

PROBLEM SOLVING

1s ≈ 10^8 Operations

- n > 10^8 O(log n) 0(1)
- n ≤ 10^8 O(n)
- n ≤ 10^6 O(n log n)
- n ≤ 10^4 O(n^2)
- n ≤ 500 O(n^3)